Hands-On Lab: Programming with Delegates and Events in C#

Objective

In this lab, you will learn how to use delegates and events in C# by building a simple stock price notification system. The system will:

1. Allow users to subscribe to stock price updates.
2. Notify users when stock prices change.

By the end of this lab, you will understand how to define and use delegates and events to build a publish-subscribe mechanism in C#.

Prerequisites

Before starting, ensure you have:

- A basic understanding of C# classes and methods.
- Familiarity with concepts like `Action` and lambda expressions is helpful but not required.

Step 1: Define the Stock Class

The `Stock` class represents a stock and raises an event when its price changes.

Instructions

1. Create a new class called `Stock`.
2. Add a property for the stock symbol and a private field for the stock price.
3. Define a custom delegate `PriceChangedHandler` to handle price change events.
4. Add an event `PriceChanged` using the custom delegate.
5. Raise the `PriceChanged` event when the price is updated.

Code

```
public class Stock
{
    public string Symbol { get; private set; }
    private decimal price;
```

```
    public decimal Price
    {
        get => price;
        set
        {
            if (price != value)
            {
                decimal oldPrice = price;
                price = value;
                OnPriceChanged(oldPrice);
            }
        }
    }

    public Stock(string symbol, decimal initialPrice)
    {
        Symbol = symbol;
        Price = initialPrice;
    }

    // Define a delegate for the PriceChanged event
    public delegate void PriceChangedHandler(Stock stock, decimal oldPrice);

    // Declare the PriceChanged event
    public event PriceChangedHandler? PriceChanged;

    // Method to raise the PriceChanged event
    protected virtual void OnPriceChanged(decimal oldPrice)
    {
        PriceChanged?.Invoke(this, oldPrice);
    }
}
```

Step 2: Create the StockWatcher Class

The `StockWatcher` class represents a user who subscribes to stock updates.

Instructions

1. Create a new class called `StockWatcher`.
2. Add a method `Subscribe` to attach to a stock's `PriceChanged` event.
3. Add a method `Unsubscribe` to detach from the event.
4. Implement a method `HandlePriceChanged` to handle event notifications.

Code

```
public class StockWatcher
{
    private readonly string name;

    public StockWatcher(string name)
    {
        this.name = name;
    }

    public void Subscribe(Stock stock)
    {
        stock.PriceChanged += HandlePriceChanged;
    }

    public void Unsubscribe(Stock stock)
    {
        stock.PriceChanged -= HandlePriceChanged;
    }

    private void HandlePriceChanged(Stock stock, decimal oldPrice)
    {
        Console.WriteLine($"{name} notified: {stock.Symbol} price changed from {oldPrice:C}
to {stock.Price:C}");
    }
}
```

Step 3: Test the System

Now, you will write a program to test the `Stock` and `StockWatcher` classes.

Instructions

1. Create a new `Program` class with a `Main` method.
2. Instantiate a few `Stock` objects.
3. Create `StockWatcher` objects and subscribe them to specific stocks.
4. Update stock prices and observe notifications.

Code

```
using System;

class Program
{
    static void Main()
```

```
    {
        var appleStock = new Stock("AAPL", 150.00m);
        var teslaStock = new Stock("TSLA", 700.00m);

        var investor1 = new StockWatcher("Investor 1");
        var investor2 = new StockWatcher("Investor 2");

        investor1.Subscribe(appleStock);
        investor2.Subscribe(teslaStock);

        Console.WriteLine("Updating stock prices...");
        appleStock.Price = 155.00m;
        teslaStock.Price = 710.00m;

        Console.WriteLine("Unsubscribing Investor 1 from AAPL...");
        investor1.Unsubscribe(appleStock);

        appleStock.Price = 160.00m;
    }
}
```

Step 4: Add Enhancements (Optional Exercises)

Exercise 1: Low Price Alerts

Add an event to the `Stock` class to notify when the price drops below a specific threshold.

Exercise 2: Track Multiple Stocks

Modify the `StockWatcher` class to subscribe to multiple stocks and display updates for all of them.

Exercise 3: Use `Action` Delegate

Refactor the `PriceChanged` event to use `Action<Stock, decimal>` as the delegate type.

Summary

- **Delegates**: Used to define a type-safe method signature for events.
- **Events**: Enable a publish-subscribe model where one object raises events, and others handle them.
- **Usage**: Use `+=` to subscribe to events and `-=` to unsubscribe.